Corporación Universitaria del Huila – Corhuila

Semana 13: Backend final y conectividad

Presentado por:

Carol Liceth Cruz Roa

Presentado a:

Jesus Ariel Gonzalez Bonilla

Facultad de Ingeniería

VI Semestre

Programación Movil

Neiva - Huila

2026

1. Descripcion del proyecto

- En la Semana 13 se implemento la conectividad real entre la app movil FitTrack U y un backend REST API desarrollado con Node.js y Express, desplegado en un contenedor Docker. Se consumen dos endpoints reales: GET para listar rutinas y POST para crear nuevas rutinas desde el formulario de la app.

2. Arquitectura del sistema

El sistema esta compuesto por dos partes que se comunican via HTTP:

- App Android (Kotlin) — cliente movil que consume la API
- Backend Node.js + Express — servidor REST corriendo en Docker

Flujo de comunicación

```
App Android (Kotlin)
    |   HTTP via OkHttp
    v
10.0.2.2:3000   (IP especial del emulador Android)
    |
Docker Container (fittracku-api)
    |
    V
Node.js + Express API
    |
    v
Datos en memoria (lista de rutinas)
```

Nota: en el emulador Android, la IP 10.0.2.2 apunta a localhost de la PC donde corre Docker.

3. Backend — Node.js + Express + Docker

3.1 Tecnologías utilizadas

- Node.js 18 (Alpine) — entorno de ejecucion JavaScript
- Express 4 — framework web para crear la API REST
- CORS — middleware para permitir peticiones desde la app movil
- Docker — contenedor para desplegar el backend

3.2 Estructura del proyecto backend

Archivo	Descripcion
src/server.js	Punto de entrada — configura Express, CORS y rutas
src/controllers/rutinas.controller.js	Logica de negocio — funciones GET, POST, DELETE
src/routes/rutinas.routes.js	Definicion de rutas de la API
Dockerfile	Instrucciones para construir la imagen Docker
.dockerignore	Archivos excluidos del contenedor
package.json	Dependencias y scripts del proyecto

3.3 Endpoints disponibles

Metodo	Endpoint	Descripcion	Respuesta
GET	/api/health	Verificar servidor activo	200 OK
GET	/api/rutinas	Listar todas las rutinas	200 + JSON array
POST	/api/rutinas	Crear nueva rutina	201 + JSON objeto
GET	/api/rutinas/:id	Obtener rutina por ID	200 + JSON objeto
DELETE	/api/rutinas/:id	Eliminar rutina	200 + mensaje

3.4 Configuracion CORS

- El backend está configurado para aceptar peticiones desde cualquier origen, lo que permite que el emulador Android se conecte sin restricciones:

```
app.use(cors({
  origin: (origin, callback) => {
    if (!origin) return callback(null, true);
    return callback(null, true);
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

3.5 Pasos para correr el backend con Docker

1. Abrir PowerShell y navegar a la carpeta FitTrackU-Backend

2. Construir la imagen Docker:
   docker build -t fittracku-backend .

3. Correr el contenedor:
   docker run -d -p 3000:3000 --name fittracku-api fittracku-backend

4. Verificar que funciona abriendo en el navegador:
   http://localhost:3000/api/health
   http://localhost:3000/api/rutinas

4. App Android — Cambios Semana 13

4.1 Dependencia agregada

Se agrego OkHttp como cliente HTTP en build.gradle.kts:
implementation("com.squareup.okhttp3:okhttp:4.12.0")

4.2 Permisos agregados en AndroidManifest.xml

<uses-permission android:name="android.permission.INTERNET"/>
android:usesCleartextTraffic="true"
El permiso INTERNET es obligatorio para hacer peticiones HTTP. El atributo usesCleartextTraffic permite conexiones HTTP (no HTTPS) necesarias para el emulador.

4.3 ApiService.kt — Cliente HTTP reutilizable

Se creo la clase ApiService que centraliza toda la comunicación con el backend. Sigue el principio de Responsabilidad Única (SRP) — solo se encarga de las peticiones HTTP:

- baseUrl centralizada: http://10.0.2.2:3000/api
- Metodo getRutinas(): hace GET /api/rutinas y devuelve lista
- Metodo createRutina(): hace POST /api/rutinas con los datos del formulario
- Manejo de errores: retorna lista vacia o false si falla la conexion

4.4 RutinasApiActivity.kt — Pantalla con estados UI

Se creo una nueva pantalla que consume el backend y muestra los 4 estados requeridos:

Estado	Descripcion	Color
Loading	ProgressBar visible + mensaje Cargando rutinas...	Gris #AAAAAA
Exito	Lista de rutinas con contador de resultados	Verde #4CAF50
Error	Mensaje claro + boton Recargar desde servidor	Rojo #E94560
Vacio	Mensaje cuando no hay rutinas disponibles	Blanco #FFFFFF

4.5 NuevaRutinaActivity.kt — Actualizado

Se actualizo el formulario para que al guardar una rutina, ademas de registrarla localmente, la envie al backend via POST /api/rutinas. El boton muestra el estado Guardando... mientras espera la respuesta del servidor.

4.6 Archivos nuevos en Android

Archivo	Tipo	Descripcion
ApiService.kt	Kotlin	Cliente HTTP reutilizable con OkHttp
RutinasApiActivity.kt	Kotlin	Pantalla que consume el backend
activity_rutinas_api.xml	XML Layout	UI con estados loading, error, vacío, éxito

5. Flujo completo de la integracion

5.1 Flujo GET — Cargar rutinas

5. Usuario toca boton Rutinas del Servidor en la pantalla principal
6. App muestra estado Loading (ProgressBar + mensaje)
7. ApiService hace GET http://10.0.2.2:3000/api/rutinas
8. Backend recibe la peticion y devuelve JSON con la lista
9. App parsea el JSON y construye las cards dinamicamente
10. Se muestra estado Exito con el numero de rutinas cargadas

5.2 Flujo POST — Crear rutina

11. Usuario llena el formulario (nombre, dia, ejercicios)
12. Toca boton Guardar Rutina — boton cambia a Guardando...
13. ApiService hace POST http://10.0.2.2:3000/api/rutinas con JSON
14. Backend valida los datos y crea la rutina en memoria
15. Si exito (201): Toast de confirmacion y regresa a pantalla anterior
16. Si error: muestra mensaje rojo en el formulario

5.3 Flujo de error

17. Si el contenedor Docker no esta corriendo
18. La peticion HTTP falla con excepcion de conexion
19. ApiService captura el error y retorna lista vacia o false
20. La pantalla muestra estado Error con mensaje claro
21. El usuario puede tocar Recargar para reintentar

6. Checklist Semana 13

Criterio	Estado
URL centralizada en ApiService (no hardcode en cada pantalla)	**Cumplido**
Cliente HTTP reutilizable con OkHttp (SRP)	**Cumplido**
GET /api/rutinas funcionando desde la app	**Cumplido**
POST /api/rutinas funcionando desde el formulario	**Cumplido**
Estado loading implementado con ProgressBar	**Cumplido**
Estado exito implementado con lista dinamica	**Cumplido**
Estado error implementado con mensaje y reintento	**Cumplido**
Estado vacio implementado	**Cumplido**
CORS configurado en el backend	**Cumplido**
Backend desplegado en contenedor Docker	**Cumplido**
Permiso INTERNET en AndroidManifest.xml	**Cumplido**
usesCleartextTraffic habilitado para HTTP	**Cumplido**
Backend probado en navegador (health + rutinas)	**Cumplido**

7. Estructura completa del proyecto

Backend

```
FitTrackU-Backend/
  src/
    controllers/
      rutinas.controller.js
    routes/
      rutinas.routes.js
    server.js
  Dockerfile
  .dockerignore
  package.json
```

Android

```
app/src/main/
  java/com/example/fittracku/
    MainActivity.kt
    HomeActivity.kt
    NuevaRutinaActivity.kt
    CamaraActivity.kt
    MapaActivity.kt
    ApiService.kt          <- NUEVO
    RutinasApiActivity.kt  <- NUEVO
  res/layout/
    activity_main.xml
    activity_home.xml
    activity_nueva_rutina.xml
    activity_camara.xml
    activity_mapa.xml
    activity_rutinas_api.xml  <- NUEVO
```